

R code and output for Appendix S1

2025-12-08

```
knitr::opts_chunk$set(echo = TRUE) # or whatever options you prefer
options(digits = 3)
```

```
library(dplyr)
library(stringr)
library(tidyr)
library(ggplot2)
library(cowplot)
library(brms)
library(readxl)
library(knitr)
library(kableExtra)
library(tidyr)
library(vegan)
library(purrr)
```

Get data from DanielSoutoV et al.

Data downloaded from: <https://doi.org/10.5281/zenodo.16586355>

Load the data and the site metadata

```
BIN_table <- read.csv("../Data/DanielSoutoV-MEE/Table S6 BIN table.csv")

metadata <- read.csv("../Data/DanielSoutoV-MEE/metadata.csv")

head(BIN_table[,1:5])
```

```

      bin_uri
1 BOLD:AAF6555
2 BOLD:AAJ4137
3 BOLD:AAK4887
4 BOLD:AAL6306
5 BOLD:AAM0757
6 BOLD:AAM9298

```

```

1      k__Animalia;p__Arthropoda;c__Insecta;o__Hymenoptera;f__Formicidae;g__Pheidoloidea;s__Formicidae;
2 k__Animalia;p__Arthropoda;c__Insecta;o__Lepidoptera;f__Tortricidae;g__Sparganothina;s__Sparganothina;
3      k__Animalia;p__Arthropoda;c__Insecta;o__Hymenoptera;f__Formicidae;g__Neoponera;s__Neoponera;
4      k__Animalia;p__Arthropoda;c__Arachnida;o__Araneae;f__Uloboridae;g__Philoponella;s__Philoponella;
5      k__Animalia;p__Arthropoda;c__Arachnida;o__Araneae;f__Araneidae;g__Parawixia;s__Parawixia;
6 k__Animalia;p__Arthropoda;c__Insecta;o__Lepidoptera;f__Notodontidae;g__NotodontidaeGEN;s__NotodontidaeGEN
      BE_P01_DRY_total BE_P01_WET_total BE_P02_DRY_total
1              1              0              0
2              1              0              0
3              1              0              0
4              1              1              0
5              1              0              0
6              1              1              1

```

Re-shape the data and the aggregate by the number of bins collected for each order vs. those collected by each trapping method

```

## reshape the data
long_bin <- BIN_table %>%
  mutate(Order = str_extract(classification, "o__[A-Z][a-z]*")) %>%
  mutate (Class = str_extract(classification, "c__[A-Z][a-z]*")) %>%
  filter(Class == "c__Insecta") %>%
  select(-classification, -total_count, -Class) %>%
  pivot_longer(values_to = "abundance", names_to = "sample", -c(bin_uri, Order)) %>%
  filter(abundance > 0) %>%
  left_join(select(metadata, sample, trap_name)) %>%
  mutate(Order = str_remove(Order, "o__")) %>%
  select(-sample) %>% unique()

```

Joining with `by = join\_by(sample)`

```

## get bin counts for malaise trapping
trap_counts <- long_bin %>%

```

```

group_by(Order, trap_name) %>%
count() %>%
filter(trap_name == 'Malaise trap')

## get bin counts for each order
order_counts <- long_bin %>%
  select(bin_uri, Order) %>%
  unique() %>%
  group_by(Order) %>%
  count() %>%
  rename(order_total = n)

## join and calculate the percentage of bins coming from malaise trapping
percentage_count <- order_counts %>%
  left_join(trap_counts) %>%
  mutate(n = ifelse(is.na(n), 0, n)) %>%
  mutate(percentage = n/order_total) %>%
  select(-trap_name, number_bins_malaise = n, number_bins_order = order_total) %>%
  mutate(Species_Unique_Other_methods = number_bins_order - number_bins_malaise)

```

Joining with `by = join\_by(Order)`

```
percentage_count
```

```

# A tibble: 19 x 5
# Groups:   Order [19]
  Order number_bins_order number_bins_malaise percentage Species_Unique_Other~1
  <chr>          <int>          <dbl>         <dbl>          <dbl>
1 Arch~             1             1           1             0
2 Blat~           119             41        0.345           78
3 Cole~          1413            551        0.390          862
4 Derm~             5             2           0.4             3
5 Dipt~          3611          1565        0.433         2046
6 Embi~             1             0           0             1
7 Ephe~             2             0           0             2
8 Hemi~           514            193        0.375           321
9 Hyme~           834            462        0.554           372
10 Lepi~          2733            383        0.140         2350
11 Mant~             3             1        0.333             2
12 Neur~            19             8        0.421            11
13 Odon~             5             1         0.2             4

```

14 Orth~	67	32	0.478	35
15 Psoc~	149	72	0.483	77
16 Stre~	2	2	1	0
17 Thys~	33	7	0.212	26
18 Tric~	44	3	0.0682	41
19 Zora~	1	0	0	1

i abbreviated name: 1: Species_Unique_Other_methods

```
write.csv(percentage_count, "../summary_percent_malaise.csv", row.names = FALSE)
```

Multi-level model to estimate probability of malaise catchability overall and by order

Fit multi-level intercept model to calculate intercept for each order

```
mm_bin <- brm(data = percentage_count,
  family = binomial,
  number_bins_malaise | trials(number_bins_order) ~ 1 + (1 | Order),
  prior = c(prior(normal(0, 1.5), class = Intercept), # alpha bar
    prior(exponential(1), class = sd)), # sigma
  iter = 5000, warmup = 1000, chains = 4, cores = 4,
  sample_prior = "yes",
  seed = 13)
```

Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c

using C compiler: 'Apple clang version 17.0.0 (clang-1700.3.19.1)'

using SDK: 'MacOSX26.0.sdk'

clang -arch arm64 -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG -I"/Users

In file included from <built-in>:1:

In file included from /Users/lmg343/Library/R/arm64/4.4/library/StanHeaders/include/stan/matl

In file included from /Users/lmg343/Library/R/arm64/4.4/library/RcppEigen/include/Eigen/Dense

In file included from /Users/lmg343/Library/R/arm64/4.4/library/RcppEigen/include/Eigen/Core

/Users/lmg343/Library/R/arm64/4.4/library/RcppEigen/include/Eigen/src/Core/util/Macros.h:679

679 | #include <cmath>

| ~~~~~~

1 error generated.

make: *** [foo.o] Error 1

Check the model summary

```
summary(mm_bin)
```

```
Family: binomial
Links: mu = logit
Formula: number_bins_malaise | trials(number_bins_order) ~ 1 + (1 | Order)
Data: percentage_count (Number of observations: 19)
Draws: 4 chains, each with iter = 5000; warmup = 1000; thin = 1;
       total post-warmup draws = 16000
```

Multilevel Hyperparameters:

```
~Order (Number of levels: 19)
      Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(Intercept)    0.78    0.20    0.48    1.26 1.00    4891    7167
```

Regression Coefficients:

```
      Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept   -0.65    0.23   -1.12   -0.20 1.00    3471    6210
```

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

Get posterior distributions, estimate means and CI, and transform back to probabilities.

Note that post_mdn, post_Q2.5, post_Q97.5, are the mean and credible interval probabilities after transformation back to proportion, while Estimate, Q2.5, and Q97.5 are in the linear scale.

```
## get posterior draws
post <- as_draws_df(mm_bin)

## Join posterior for each order with original counts
post_mdn <-
  coef(mm_bin, robust = T)$Order[, , ] %>%
  data.frame() %>%
  bind_cols(select(percentage_count, Order,
                  number_bins_order, number_bins_malaise)) %>%
  mutate(post_mdn = inv_logit_scaled(Estimate)) %>%
  mutate(post_Q2.5 = inv_logit_scaled(Q2.5),
         post_Q97.5 = inv_logit_scaled(Q97.5)) %>%
  mutate(percentage_raw = number_bins_malaise/number_bins_order) %>%
  select(Order, number_bins_order, number_bins_malaise,
```

```
percentage_raw, starts_with('post'),
Estimate:`Q97.5`)
```

```
rownames(post_mdn) <- NULL
```

```
post_mdn
```

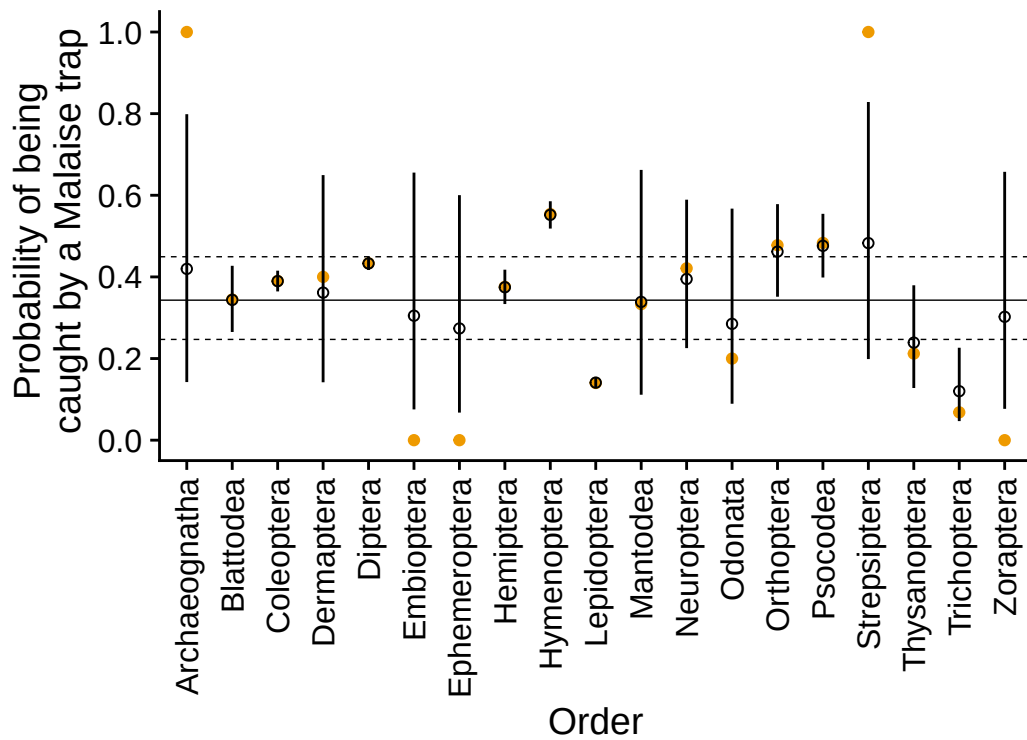
	Order	number_bins_order	number_bins_malaise	percentage_raw	post_mdn	
1	Archaeognatha	1	1	1.0000	0.420	
2	Blattodea	119	41	0.3445	0.344	
3	Coleoptera	1413	551	0.3900	0.390	
4	Dermaptera	5	2	0.4000	0.362	
5	Diptera	3611	1565	0.4334	0.433	
6	Embioptera	1	0	0.0000	0.305	
7	Ephemeroptera	2	0	0.0000	0.274	
8	Hemiptera	514	193	0.3755	0.375	
9	Hymenoptera	834	462	0.5540	0.552	
10	Lepidoptera	2733	383	0.1401	0.141	
11	Mantodea	3	1	0.3333	0.339	
12	Neuroptera	19	8	0.4211	0.395	
13	Odonata	5	1	0.2000	0.285	
14	Orthoptera	67	32	0.4776	0.462	
15	Psocodea	149	72	0.4832	0.476	
16	Strepsiptera	2	2	1.0000	0.483	
17	Thysanoptera	33	7	0.2121	0.239	
18	Trichoptera	44	3	0.0682	0.120	
19	Zoraptera	1	0	0.0000	0.302	
	post_Q2.5	post_Q97.5	Estimate	Est.Error	Q2.5	Q97.5
1	0.1424	0.799	-0.3241	0.7426	-1.7956	1.378
2	0.2651	0.427	-0.6470	0.1847	-1.0198	-0.293
3	0.3644	0.415	-0.4486	0.0539	-0.5562	-0.342
4	0.1419	0.649	-0.5688	0.5887	-1.7998	0.616
5	0.4169	0.449	-0.2695	0.0333	-0.3354	-0.203
6	0.0754	0.656	-0.8238	0.7351	-2.5071	0.643
7	0.0676	0.600	-0.9757	0.7149	-2.6242	0.407
8	0.3336	0.418	-0.5123	0.0908	-0.6921	-0.332
9	0.5184	0.585	0.2090	0.0687	0.0737	0.345
10	0.1280	0.154	-1.8072	0.0547	-1.9190	-1.701
11	0.1114	0.662	-0.6686	0.6548	-2.0769	0.673
12	0.2256	0.589	-0.4270	0.4062	-1.2336	0.361
13	0.0892	0.567	-0.9196	0.6207	-2.3229	0.271
14	0.3514	0.578	-0.1526	0.2337	-0.6128	0.316

15	0.3986	0.555	-0.0955	0.1607	-0.4114	0.219
16	0.1987	0.828	-0.0687	0.7040	-1.3945	1.575
17	0.1278	0.379	-1.1579	0.3657	-1.9206	-0.492
18	0.0468	0.226	-1.9927	0.4501	-3.0149	-1.228
19	0.0768	0.657	-0.8366	0.7390	-2.4870	0.652

Visualize new results

```
probability_caught_malaise_fig <- post_mdn %>%
  ggplot(aes(x = Order)) +
  geom_hline(yintercept = inv_logit_scaled(mean(post$b_Intercept)),
    linetype = 1, linewidth = 1/4) +
  geom_hline(yintercept = inv_logit_scaled(quantile(post$b_Intercept, 0.025)),
    linetype = 2, linewidth = 1/4) +
  geom_hline(yintercept = inv_logit_scaled(quantile(post$b_Intercept, 0.975)),
    linetype = 2, linewidth = 1/4) +
  geom_point(aes(y = percentage_raw), color = "orange2") +
  geom_point(aes(y = post_mdn), shape = 1) +
  geom_linerange(aes(ymin = post_Q2.5, ymax=post_Q97.5))+
  scale_y_continuous(breaks = 0:5 / 5, limits = c(0, 1)) +
  theme_cowplot() +
  ylab('Probability of being \n caught by a Malaise trap') +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1))

probability_caught_malaise_fig
```



```
ggsave(probability_caught_malaise_fig, filename = "../Figures/probability_caught_malaise_fig.png")
```

Saving 5.5 x 4 in image

Get overall level mean and CI

```
inv_logit_scaled(mean(post$b_Intercept))
```

```
[1] 0.343
```

```
inv_logit_scaled(quantile(post$b_Intercept, 0.025))
```

```
2.5%
0.247
```

```
inv_logit_scaled(quantile(post$b_Intercept, 0.975))
```

```
97.5%
0.449
```

Incorporate Number of BINS collected in other orders

For each Order in the BCI data, we divide the records into *Malaise* and *Others* (the other six sampling methods), and then:

1. **Compute** how many species are unique to *Others* — where “unique” means species captured only by the other methods, not by Malaise.
2. **Compute the proportion** = (# of unique species in *Others*) / (# of Malaise species). This proportion roughly represents the rate at which Malaise fails to capture species that could be obtained if all other six sampling methods were applied to our core data.
3. **Adjust** our core data by multiplying the number of species in the core data by (1+ proportion).
4. **Repeat** this adjustment Order by Order for *all* Orders. This simple adjustment can be applied even to sparse Orders, so we need not worry about data sufficiency.
5. As a result, the total number of species (currently 53,945 in our core Malaise dataset) will increase somewhat, and the final estimate will inevitably be larger.

```
## Compute how many species are unique to Others -  
#where "unique" means species captured only by the other methods, not by Malaise.  
  
malaise_bins <- long_bin %>%  
  filter(abundance > 0) %>%  
  select(bin_uri, Order, trap_name) %>%  
  unique() %>%  
  filter(trap_name == 'Malaise trap')  
  
non_malaise_bins <- long_bin %>%  
  filter(abundance > 0) %>%  
  select(bin_uri, Order) %>%  
  unique() %>%  
  filter(!bin_uri %in% malaise_bins$bin_uri)  
  
unique_non_malaise <- non_malaise_bins %>%  
  group_by(Order) %>%  
  count()  
  
kable(unique_non_malaise)
```

Order	n
Blattodea	78
Coleoptera	862
Dermaptera	3
Diptera	2046
Embioptera	1
Ephemeroptera	2
Hemiptera	321
Hymenoptera	372
Lepidoptera	2350
Mantodea	2
Neuroptera	11
Odonata	4
Orthoptera	35
Psocodea	77
Thysanoptera	26
Trichoptera	41
Zoraptera	1

#Compute the proportion = (# of unique species in Others) / (# of Malaise species).

```
n_total <- long_bin %>%
  filter(abundance > 0) %>%
  select(bin_uri, Order) %>%
  unique() %>%
  group_by(Order) %>%
  count() %>%
  rename(n_bins_total = n)

ratio_unique_others <- malaise_bins %>%
  group_by(Order) %>%
  count() %>%
  rename(n_malaise_bins = n) %>%
  full_join(unique_non_malaise) %>%
  mutate(n = ifelse(is.na(n), 0, n)) %>%
  rename(n_unique_non_malaise = n) %>%
  mutate(n_malaise_bins = ifelse(is.na(n_malaise_bins), 0, n_malaise_bins)) %>%
  mutate(ratio = (n_unique_non_malaise/(n_malaise_bins))) %>%
  left_join(n_total) %>%
  select(n_bins_total, n_malaise_bins, n_unique_non_malaise, ratio) %>%
  arrange(Order) %>%
```

```
mutate(ratio = ifelse(is.infinite(ratio), 0, ratio))
```

Joining with `by = join\_by(Order)`
 Joining with `by = join\_by(Order)`
 Adding missing grouping variables: `Order`

```
kable(ratio_unique_others)
```

Order	n_bins_total	n_malaise_bins	n_unique_non_malaise	ratio
Archaeognatha	1	1	0	0.000
Blattodea	119	41	78	1.902
Coleoptera	1413	551	862	1.564
Dermaptera	5	2	3	1.500
Diptera	3611	1565	2046	1.307
Embioptera	1	0	1	0.000
Ephemeroptera	2	0	2	0.000
Hemiptera	514	193	321	1.663
Hymenoptera	834	462	372	0.805
Lepidoptera	2733	383	2350	6.136
Mantodea	3	1	2	2.000
Neuroptera	19	8	11	1.375
Odonata	5	1	4	4.000
Orthoptera	67	32	35	1.094
Psocodea	149	72	77	1.069
Strepsiptera	2	2	0	0.000
Thysanoptera	33	7	26	3.714
Trichoptera	44	3	41	13.667
Zoraptera	1	0	1	0.000

```
#Adjust our core data by multiplying the
#number of species in the core data by (1+ proportion).

Order_richness_core <- read_xlsx("../Data/Insect_Orders_count.xlsx") %>%
  rename(Order = order)

adjusted_core <- ratio_unique_others %>%
  full_join(Order_richness_core) %>%
  arrange(Order) %>%
  mutate_if(is.numeric, coalesce, 0) %>%
```

```
mutate(adjusted_richness = distinctBINS * (ratio+1)) %>%
rename(Bins_ACG = distinctBINS)
```

Joining with `by = join\_by(Order)`
`mutate\_if()` ignored the following grouping variables:

```
kable(adjusted_core)
```

Order	n_bins_total	n_malaise_bins	n_unique_non_malaise	ratio	Bins_ACG	adjusted_richness
Archaeognatha	1	1	0	0.000	18	18.0
Blattodea	119	41	78	1.902	186	539.9
Coleoptera	1413	551	862	1.564	6844	17550.9
Dermaptera	5	2	3	1.500	31	77.5
Diptera	3611	1565	2046	1.307	24528	56594.6
Embioptera	1	0	1	0.000	4	4.0
Ephemeroptera	2	0	2	0.000	7	7.0
Hemiptera	514	193	321	1.663	2679	7134.7
Hymenoptera	834	462	372	0.805	13934	25153.6
Lepidoptera	2733	383	2350	6.136	4455	31789.9
Mantodea	3	1	2	2.000	23	69.0
Mecoptera	0	0	0	0.000	1	1.0
Neuroptera	19	8	11	1.375	86	204.2
Odonata	5	1	4	4.000	1	5.0
Orthoptera	67	32	35	1.094	320	670.0
Phasmatodea	0	0	0	0.000	18	18.0
Plecoptera	0	0	0	0.000	8	8.0
Psocodea	149	72	77	1.069	297	614.6
Strepsiptera	2	2	0	0.000	3	3.0
Thysanoptera	33	7	26	3.714	286	1348.3
Trichoptera	44	3	41	13.667	210	3080.0
Zoraptera	1	0	1	0.000	2	2.0
Zygentoma	0	0	0	0.000	5	5.0

```
sum(adjusted_core$adjusted_richness)
```

```
[1] 144898
```

Stability of the ratio (BCI-Unique-to-Other)/(BCI Malaise) by rarefaction

- (1) Choose one of the 50 **Malaise** samples, at random **without replacement**. Keep its list of **N** species names (or codes) captured in the first column of an **N-by-50** matrix called something like **Accumulated Malaise**.
- (2) Choose one of the 50 **non-Malaise** samples, for **each of the 7 non-Malaise** collection methods, at random **without replacement**. Pool the 7 samples of species captured and place them in the first column of an **N-by-50** matrix called something like **Accumulated Non-Malaise**.
- (3) Compare the **Accumulated Malaise** with **Accumulated Non-Malaise** species lists and delete any **matches** from the **Non-Malaise** list, to create the first column of an **N-by-50** matrix called something like **Accumulated Unique to Non-Malaise**.
- (4) **Repeat steps 1 to 3 (49 times)** until all 50 samples for all methods have been **accumulated**, placing the successive results in successive columns of the **N** by 50 matrices. In other words, e.g. Column **j** is the list from Column (**j - 1**) with species new to step **j** added.
- (5) Plot the **species accumulation curves** for **Malaise** and **Unique-to-Non-Malaise** (the incidence total for their successive 50 columns).
- (6) Re-run the entire simulation 100 times, and plot the **mean** curves, to get smooth curves.

```
## reshape and prepare the data
long_sample <- BIN_table %>%
  mutate(Order = str_extract(classification, "o__[A-Z][a-z]*")) %>%
  mutate(Class = str_extract(classification, "c__[A-Z][a-z]*")) %>%
  filter(Class == "c__Insecta") %>%
  select(-classification, -total_count, -Class) %>%
  pivot_longer(values_to = "abundance",
               names_to = "sample", -c(bin_uri, Order)) %>%
  filter(abundance > 0) %>%
  left_join(select(metadata, sample, trap_name)) %>%
  mutate(Order = str_remove(Order, "o__"))
```

Joining with `by = join\_by(sample)`

```
## step 1

malaise_samples_all <- long_sample %>%
  filter(trap_name == 'Malaise trap') %>%
  select(sample) %>% unique() %>% unlist()
```

```

sample_all_malaise <- sample(malaise_samples_all, 50, replace = FALSE)
## choose 50 so replace = false works

chosen_malaise_sample <- sample_all_malaise[1]

species_in_malaise_chosen_sample <- long_sample %>%
  filter(sample == chosen_malaise_sample) %>%
  select(bin_uri) %>% unique() %>% unlist()

## step 2

##### update to sample for each sampling method.

# filter for non malaise traps
non_malaise_samples_all <- long_sample %>%
  filter(trap_name != 'Malaise trap') %>%
  select(sample, trap_name) %>% unique()

# split samples by trap
samples_by_type <- split(non_malaise_samples_all, non_malaise_samples_all$trap_name)

# for each trap type sample without replacement
sample_all_non_malaise <- lapply(samples_by_type,
                                function(x) sample(x$sample, 50, replace = FALSE))
## choose 50 so replace = false works

# select the first sample for each trap method
chosen_non_malaise_sample <- sapply(sample_all_non_malaise, FUN = function(x) x[1])

chosen_non_malaise_sample

```

Beating	Berlese	Light trap	Pitfall
"BE_P10_WET_total"	"B_P24_WET_total"	"LT_P06_WET_total"	"PF_P09_WET_total"
Polytrap	Winkler		
"PT_P21_DRY_total"	"W_P17_WET_total"		

```

species_in_non_malaise_chosen_sample <- long_sample %>%
  filter(sample %in% chosen_non_malaise_sample) %>%
  select(bin_uri) %>% unique() %>% unlist()

#Step 3

```

```

unique_non_malaise <-
  species_in_non_malaise_chosen_sample[!species_in_non_malaise_chosen_sample %in%
                                         species_in_malaise_chosen_sample]

## Step 4

malaise_bin <- data.frame(bin = species_in_malaise_chosen_sample, iter = 1)
unique_non_malaise_bin <- data.frame(bin = unique_non_malaise, iter = 1)

for(i in 2:50){

  # step 1

  chosen_malaise_sample <- sample_all_malaise[i]

  species_in_malaise_chosen_sample <- long_sample %>%
    filter(sample == chosen_malaise_sample) %>%
    select(bin_uri) %>% unique() %>% unlist()

  malaise_bin <- bind_rows(malaise_bin,
                           data.frame(bin = species_in_malaise_chosen_sample,
                                       iter = i))

  # step 2

  chosen_non_malaise_sample <- sapply(sample_all_non_malaise, FUN = function(x) x[i])

  species_in_non_malaise_chosen_sample <- long_sample %>%
    filter(sample %in% chosen_non_malaise_sample) %>%
    select(bin_uri) %>% unique() %>% unlist()

  # step 3 ## remove duplicate bins just added to Malaise

  unique_non_malaise <-
    species_in_non_malaise_chosen_sample[!species_in_non_malaise_chosen_sample
                                           %in% unique(malaise_bin$bin)]

  unique_non_malaise_bin <- bind_rows(unique_non_malaise_bin,
                                       data.frame(bin = unique_non_malaise,
                                                   iter = i))
}

```

```

rownames(malaise_bin) <- NULL

rownames(unique_non_malaise_bin) <- NULL

### malaise accumulated richness

malaise_matrix <- malaise_bin %>%
  mutate(present = 1) %>%
  pivot_wider(names_from = iter, values_from = present, values_fill = 0) %>%
  tibble::column_to_rownames("bin") %>%
  t()

malaise_species_accumulation <-
  specaccum(malaise_matrix, method = 'collector')

malaise_accumulated_richness <-
  malaise_species_accumulation$richness

### unique non Malaise accumulated richness

unique_non_matrix <- unique_non_malaise_bin %>%
  mutate(present = 1) %>%
  pivot_wider(names_from = iter, values_from = present, values_fill = 0) %>%
  tibble::column_to_rownames("bin") %>%
  t()

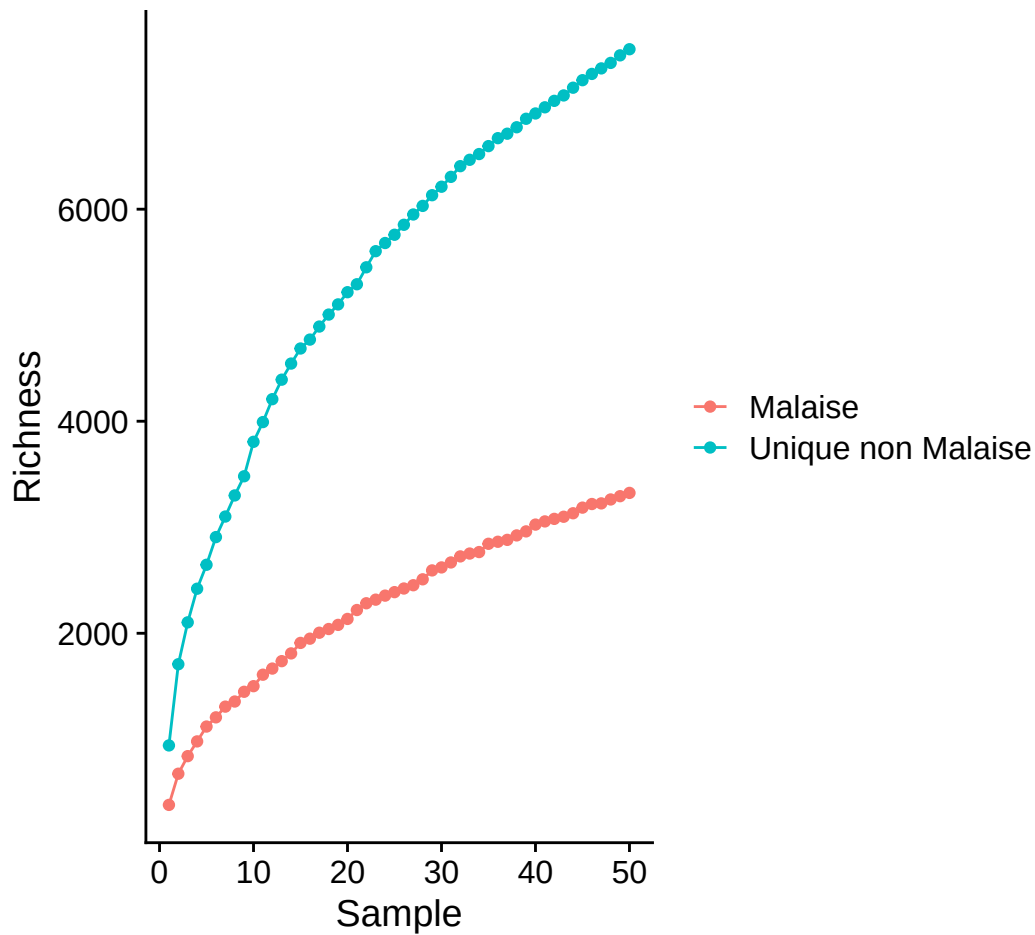
unique_non_malaise_species_accumulation <-
  specaccum(unique_non_matrix, method = 'collector')

unique_non_malaise_accumulated_richness <-
  unique_non_malaise_species_accumulation$richness

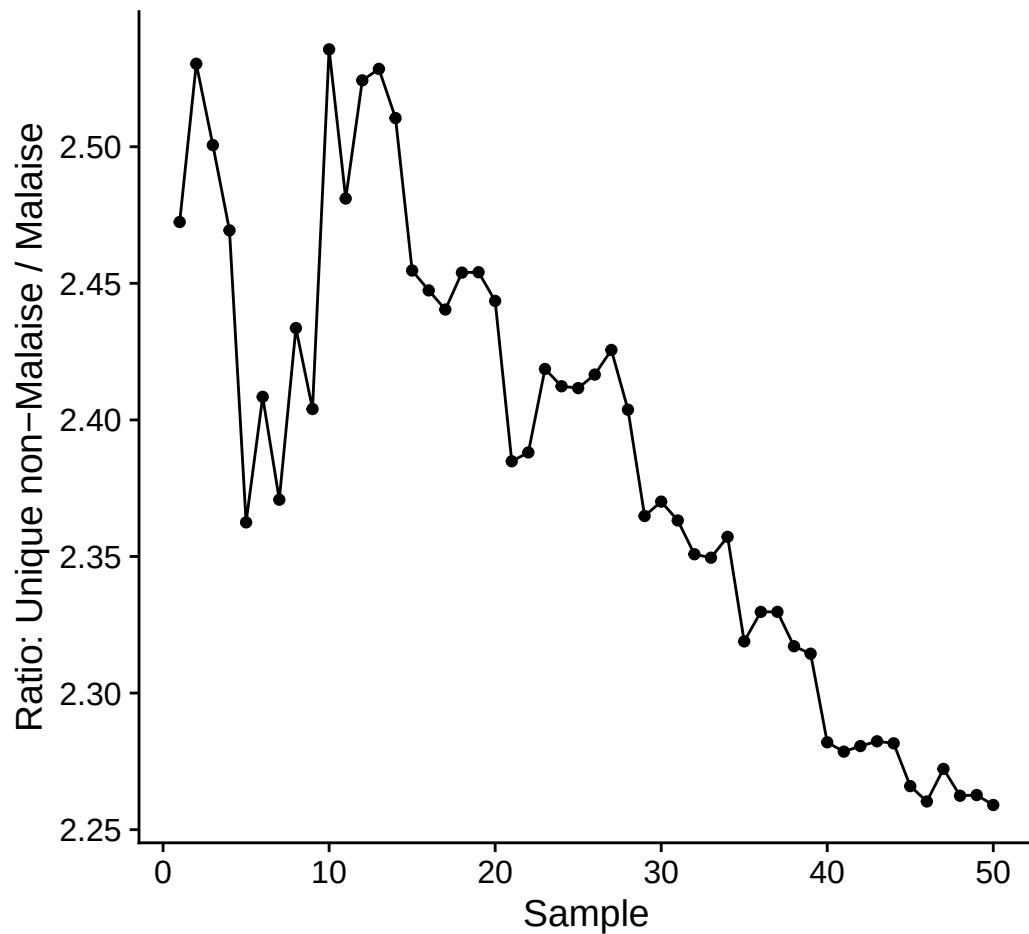
bind_rows(data.frame(richness = malaise_accumulated_richness,
                     type = "Malaise", sample = 1:50),
          data.frame(richness = unique_non_malaise_accumulated_richness,
                     type = "Unique non Malaise",
                     sample = 1:50)) %>%
  ggplot(aes(x = sample, y = richness, color = type)) +
  geom_point() +
  geom_line() +

```

```
theme_cowplot() + ylab("Richness") + xlab("Sample") +
theme(legend.title = element_blank())
```



```
data.frame(malaise_accumulated_richness,
            unique_non_malaise_accumulated_richness, sample = 1:50) %>%
mutate(ratio =
        unique_non_malaise_accumulated_richness/malaise_accumulated_richness) %>%
ggplot(aes(x = sample, y = ratio)) +
geom_point() +
geom_line()+
theme_cowplot() + ylab("Ratio: Unique non-Malaise / Malaise") + xlab("Sample")
```



Now run 100 times

```
#step 6 repeat 100 times

accumulated_malaise <- matrix(NA, nrow = 50, ncol = 100)
accumulated_unique_non_malaise <- matrix(NA, nrow = 50, ncol = 100)

for(n in 1:100){

  malaise_bin <- data.frame()

  unique_non_malaise_bin <- data.frame()

  ## sample samples
```

```

sample_all_malaise <- sample(malaise_samples_all, 50, replace = FALSE)

# split samples by trap
samples_by_type <- split(non_malaise_samples_all,
                        non_malaise_samples_all$trap_name)

# for each trap type sample without replacement
sample_all_non_malaise <- lapply(samples_by_type,
                                function(x) sample(x$sample, 50, replace = FALSE))
## choose 50 so replace = false works

## steps 1-4

for(i in 1:50){

# step 1

chosen_malaise_sample <- sample_all_malaise[i]

species_in_malaise_chosen_sample <- long_sample %>%
  filter(sample == chosen_malaise_sample) %>%
  select(bin_uri) %>% unique() %>% unlist()

malaise_bin <- bind_rows(malaise_bin,
                        data.frame(bin = species_in_malaise_chosen_sample,
                                  iter = i))

# step 2
# select the first sample for each trap method
chosen_non_malaise_sample <- sapply(sample_all_non_malaise, FUN = function(x) x[i])

species_in_non_malaise_chosen_sample <- long_sample %>%
  filter(sample %in% chosen_non_malaise_sample) %>%
  select(bin_uri) %>% unique() %>% unlist()

# step 3

unique_non_malaise <-
  species_in_non_malaise_chosen_sample[!species_in_non_malaise_chosen_sample
                                        %in% unique(malaise_bin$bin)]
if(length(unique_non_malaise) != 0){

```

```

    ## check that unique_non_malaise is not empty

    unique_non_malaise_bin <- bind_rows(unique_non_malaise_bin,
                                         data.frame(bin = unique_non_malaise,
                                                    iter = i))
  }else{
    ## if its empty, add the last bin of the previous iteration just to make sure there are
    unique_non_malaise_bin <-
      bind_rows(unique_non_malaise_bin,
                data.frame(bin =
                           unique_non_malaise_bin$bin[nrow(unique_non_malaise_bin)],
                           iter = i))

  }
}

### get the accumulation

rownames(malaise_bin) <- NULL

rownames(unique_non_malaise_bin) <- NULL

### malaise accumulated richness

malaise_matrix <- malaise_bin %>%
  mutate(present = 1) %>%
  pivot_wider(names_from = iter, values_from = present, values_fill = 0) %>%
  tibble::column_to_rownames("bin") %>%
  t()

malaise_species_accumulation <-
  specaccum(malaise_matrix, method = 'collector')

malaise_accumulated_richness <-
  malaise_species_accumulation$richness

### unique non Malaise accumulated richness

unique_non_matrix <- unique_non_malaise_bin %>%
  mutate(present = 1) %>%

```

```

pivot_wider(names_from = iter, values_from = present, values_fill = 0) %>%
tibble::column_to_rownames("bin") %>%
t()

unique_non_malaise_species_accumulation <-
  specaccum(unique_non_matrix, method = 'collector')

unique_non_malaise_accumulated_richness <-
  unique_non_malaise_species_accumulation$richness

## store results

accumulated_malaise[,n] <- malaise_accumulated_richness
accumulated_unique_non_malaise[,n] <- unique_non_malaise_accumulated_richness
}

```

Add plots

```

## mean accumulation curves

malaise_mean <- apply(accumulated_malaise, 1,
  FUN= function(x) c(mean(x), quantile(x, 0.025),
    quantile(x, 0.975)))

unique_non_malaise_mean <- apply(accumulated_unique_non_malaise, 1,
  FUN= function(x) c(mean(x), quantile(x, 0.025),
    quantile(x, 0.975)))

sample_size <- 50

sample_malaise_non_malaise_fig <- bind_rows(data.frame(t(malaise_mean), type = "Malaise", sample = 1:sample_size),
  data.frame(t(unique_non_malaise_mean),
    type = "Unique non Malaise", sample = 1:sample_size))

ggplot() +
  geom_line(aes(x = sample, y = V1, color = type)) +
  geom_ribbon(aes(x = sample, ymin = `X2.5.`, ymax = `X97.5.`, fill = type), alpha = 0.2) +
  theme_cowplot() +
  ylab("Richness") + xlab("Sample") +

```

```
theme(legend.title = element_blank()) +
theme(legend.position = c(0.05, 0.9))
```

Warning: A numeric `legend.position` argument in `theme()` was deprecated in ggplot2 3.5.0.

i Please use the `legend.position.inside` argument of `theme()` instead.

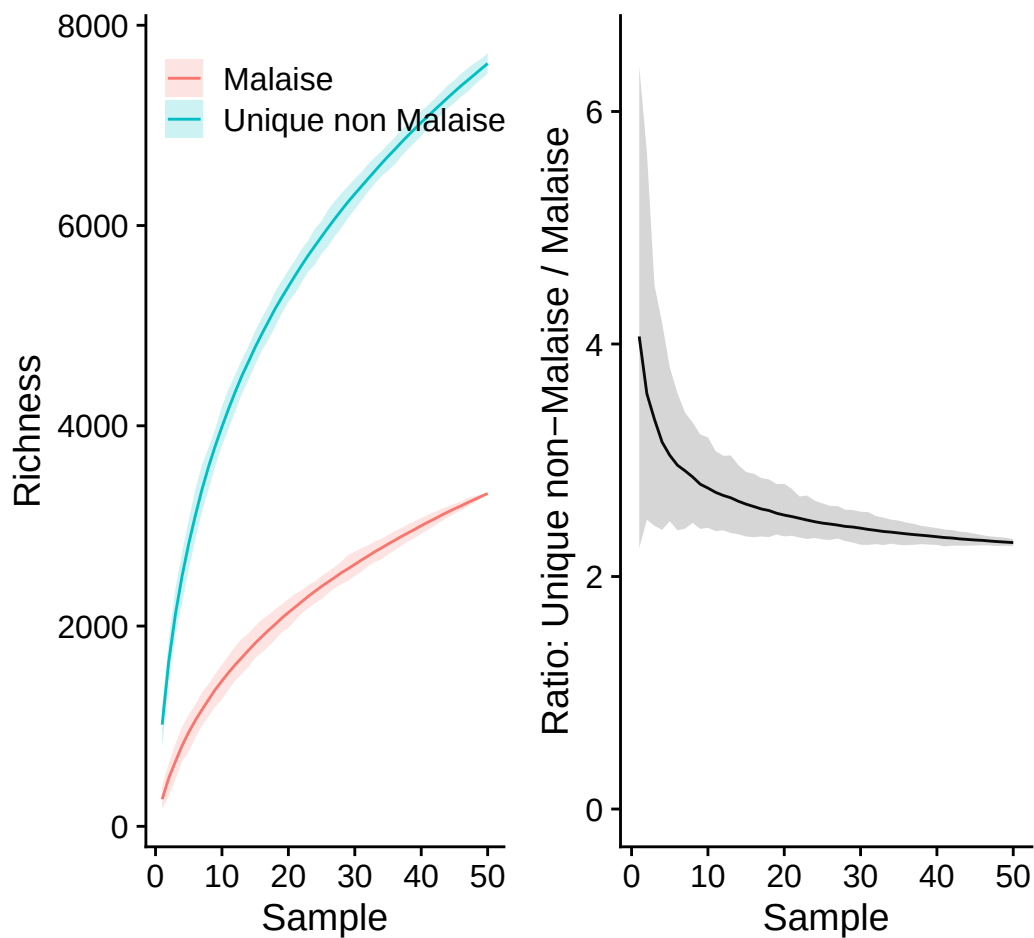
```
### mean ratio
```

```
ratio_mean <- apply(accumulated_unique_non_malaise/accumulated_malaise, 1,
  FUN= function(x) c(mean(x), quantile(x, 0.025),
    quantile(x, 0.975)))
```

```
ratio_sample_all_fig <- data.frame(t(ratio_mean), sample = 1:sample_size) %>%
  ggplot() +
  geom_line(aes(x = sample, y = V1)) +
  geom_ribbon(aes(x = sample, ymin = `X2.5.`, ymax = `X97.5.`), alpha = 0.2) +
  theme_cowplot() +
  ylab("Ratio: Unique non-Malaise / Malaise") + xlab("Sample") +
  scale_y_continuous(limits = c(0, 6.5))
```

```
combined_saturation_ratio <- plot_grid(sample_malaise_non_malaise_fig, ratio_sample_all_fig)
```

```
combined_saturation_ratio
```



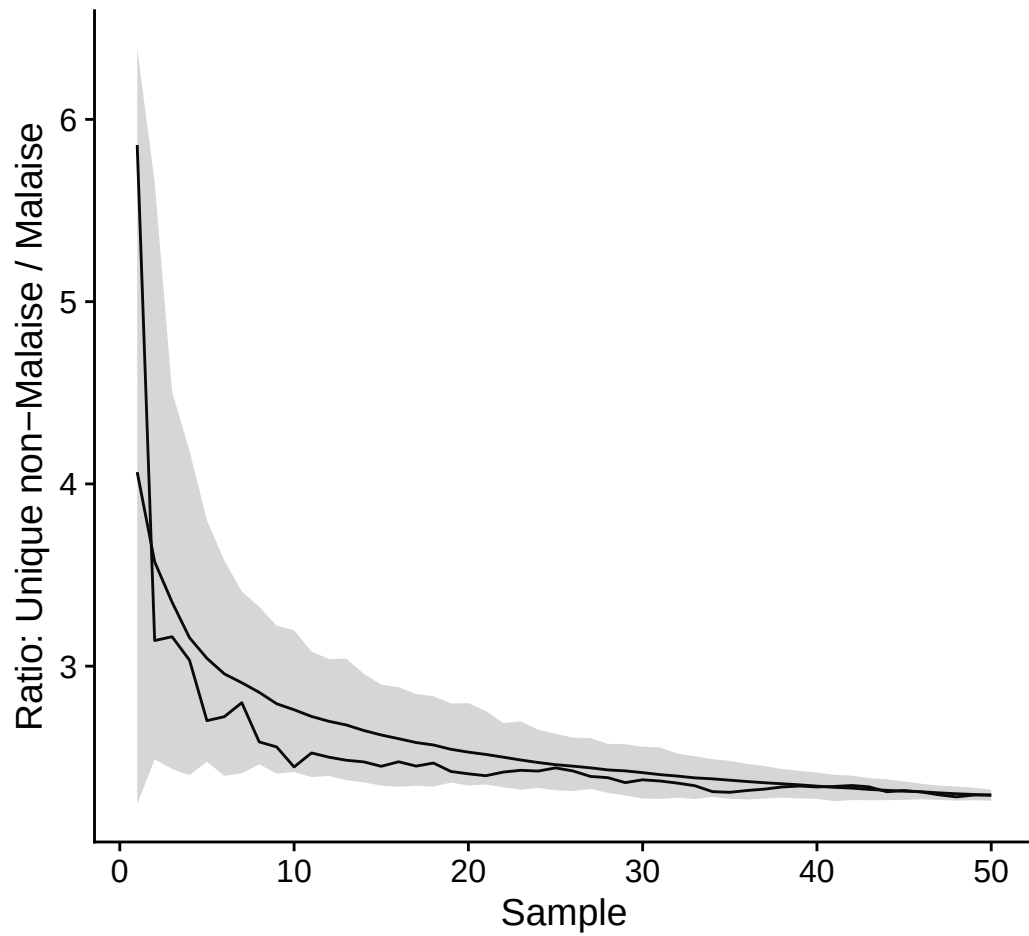
```
ggsave(combined_saturation_ratio,
       filename = paste0("../Figures/combined_saturation_ratio",sample_size,".jpeg"), width = 1000, height = 1000)

## add the one sample result

one_sample <-data.frame(malaise_accumulated_richness,
                       unique_non_malaise_accumulated_richness, sample = 1:50) %>%
  mutate(ratio = unique_non_malaise_accumulated_richness/malaise_accumulated_richness)

data.frame(t(ratio_mean), sample = 1:50) %>%
  ggplot() +
  geom_line(aes(x = sample, y = V1)) +
  geom_line(data = one_sample, aes(x = sample, y = ratio)) +
  geom_ribbon(aes(x = sample, ymin = `X2.5.`, ymax = `X97.5.`), alpha = 0.2) +
  theme_cowplot() +
```

```
ylab("Ratio: Unique non-Malaise / Malaise") + xlab("Sample")
```



```
t(ratio_mean)
```

		2.5%	97.5%
[1,]	4.06	2.24	6.39
[2,]	3.57	2.49	5.66
[3,]	3.35	2.44	4.51
[4,]	3.16	2.40	4.18
[5,]	3.04	2.48	3.80
[6,]	2.96	2.40	3.58
[7,]	2.91	2.41	3.41
[8,]	2.86	2.46	3.33
[9,]	2.79	2.41	3.22
[10,]	2.76	2.42	3.20

[11,]	2.72	2.39	3.08
[12,]	2.70	2.40	3.04
[13,]	2.68	2.37	3.04
[14,]	2.65	2.36	2.96
[15,]	2.62	2.34	2.90
[16,]	2.60	2.34	2.88
[17,]	2.58	2.34	2.85
[18,]	2.57	2.34	2.84
[19,]	2.54	2.36	2.80
[20,]	2.53	2.34	2.80
[21,]	2.52	2.35	2.75
[22,]	2.50	2.34	2.69
[23,]	2.49	2.32	2.70
[24,]	2.47	2.33	2.65
[25,]	2.46	2.32	2.63
[26,]	2.45	2.31	2.61
[27,]	2.44	2.33	2.61
[28,]	2.43	2.31	2.57
[29,]	2.43	2.29	2.57
[30,]	2.42	2.27	2.56
[31,]	2.40	2.27	2.55
[32,]	2.40	2.28	2.52
[33,]	2.39	2.27	2.51
[34,]	2.38	2.28	2.49
[35,]	2.37	2.27	2.48
[36,]	2.37	2.27	2.46
[37,]	2.36	2.27	2.45
[38,]	2.35	2.28	2.44
[39,]	2.35	2.27	2.43
[40,]	2.34	2.27	2.42
[41,]	2.34	2.26	2.40
[42,]	2.33	2.27	2.40
[43,]	2.32	2.26	2.39
[44,]	2.32	2.26	2.38
[45,]	2.31	2.27	2.37
[46,]	2.31	2.27	2.35
[47,]	2.30	2.27	2.34
[48,]	2.30	2.26	2.34
[49,]	2.30	2.26	2.33
[50,]	2.29	2.26	2.32

Suppose the BCI study had run two Malaise traps in each of the 50 sampling plots (instead of one), or ran the one trap per sampling unit for twice as many days. To a rough approxima-

tion, that would be equivalent to adding two Malaise samples for each Unique-to-Non-Malaise samples in the simulations. That would move the red curves to the left and reduce the ratio.

```
#step 6 repeat 100 times

accumulated_malaise <- matrix(NA, nrow = 25, ncol = 100)
accumulated_unique_non_malaise <- matrix(NA, nrow = 25, ncol = 100)

orders <- sort(unique(long_sample$Order))
accumulated_unique_non_malaise_order <- matrix(NA, ncol = 100, nrow = length(orders))

for(n in 1:100){

  malaise_bin <- data.frame()

  unique_non_malaise_bin <- data.frame()

  ## sample samples

  sample_all_malaise <- sample(malaise_samples_all, 50, replace = FALSE)

  # split samples by trap
  samples_by_type <- split(non_malaise_samples_all, non_malaise_samples_all$trap_name)

  # for each trap type sample without replacement
  sample_all_non_malaise <- lapply(samples_by_type,
                                   function(x) sample(x$sample, 50, replace = FALSE))

  ## steps 1-4

  for(i in 1:25){

    # step 1

    chosen_malaise_sample <- sample_all_malaise[c(i, i+25)]

    species_in_malaise_chosen_sample <- long_sample %>%
      filter(sample %in% chosen_malaise_sample) %>%
      select(bin_uri) %>% unique() %>% unlist()

    malaise_bin <- bind_rows(malaise_bin,
                             data.frame(bin = species_in_malaise_chosen_sample,
```

```

                                iter = i))

# step 2

chosen_non_malaise_sample <- sample_all_non_malaise, FUN = function(x) x[i])

species_in_non_malaise_chosen_sample <- long_sample %>%
  filter(sample %in% chosen_non_malaise_sample) %>%
  select(bin_uri) %>% unique() %>% unlist()

# step 3

unique_non_malaise <-
  species_in_non_malaise_chosen_sample[!species_in_non_malaise_chosen_sample
                                         %in% unique(malaise_bin$bin)]

if(length(unique_non_malaise) != 0){

  ## check that unique_non_malaise is not empty

  unique_non_malaise_bin <- bind_rows(unique_non_malaise_bin,
                                     data.frame(bin = unique_non_malaise,
                                               iter = i))
}else{
  ## if its empty, add the last bin of the previous
  #iteration just to make sure there are not fewer iterations,
  #and ultimately no species will get accumulated
  unique_non_malaise_bin <-
    bind_rows(unique_non_malaise_bin,
              data.frame(bin = unique_non_malaise_bin$bin[nrow(unique_non_malaise_bin)],
                        iter = i))
}

}

}

### get the accumulation

rownames(malaise_bin) <- NULL

rownames(unique_non_malaise_bin) <- NULL

```

```

### malaise accumulated richness

malaise_matrix <- malaise_bin %>%
  mutate(present = 1) %>%
  pivot_wider(names_from = iter, values_from = present, values_fill = 0) %>%
  tibble::column_to_rownames("bin") %>%
  t()

malaise_species_accumulation <-
  specaccum(malaise_matrix, method = 'collector')

malaise_accumulated_richness <-
  malaise_species_accumulation$richness

### unique non Malaise accumulated richness

unique_non_matrix <- unique_non_malaise_bin %>%
  mutate(present = 1) %>%
  pivot_wider(names_from = iter, values_from = present, values_fill = 0) %>%
  tibble::column_to_rownames("bin") %>%
  t()

unique_non_malaise_species_accumulation <-
  specaccum(unique_non_matrix, method = 'collector')

unique_non_malaise_accumulated_richness <-
  unique_non_malaise_species_accumulation$richness

## by order

bin_order <- long_sample %>%
  select(bin_uri, Order) %>%
  unique() %>% rename(bin = bin_uri)

unique_non_malaise_order <- unique_non_malaise_bin %>%
  mutate(present = 1) %>%
  left_join(bin_order)

total_bins_order <- unique_non_malaise_order %>%
  select(bin, Order) %>%
  unique() %>%

```

```

group_by(Order) %>%
count() %>%
ungroup() %>%
full_join(data.frame(Order = orders)) %>%
mutate(n = ifelse(is.na(n), 0, n)) %>%
arrange(Order)

accumulated_unique_non_malaise_order[,n] <- total_bins_order$n

## store results

accumulated_malaise[,n] <- malaise_accumulated_richness
accumulated_unique_non_malaise[,n] <- unique_non_malaise_accumulated_richness
}

```

Add plots

```

## mean accumulation curves

malaise_mean <- apply(accumulated_malaise, 1,
                      FUN= function(x) c(mean(x), quantile(x, 0.025),
                                           quantile(x, 0.975)))

unique_non_malaise_mean <- apply(accumulated_unique_non_malaise, 1,
                                FUN= function(x) c(mean(x), quantile(x, 0.025),
                                                       quantile(x, 0.975)))

sample_size <- 25

sample_malaise_non_malaise_fig <- bind_rows(data.frame(t(malaise_mean), type = "Malaise", sample = 1:sample_size),
                                             data.frame(t(unique_non_malaise_mean),
                                                         type = "Unique non Malaise", sample = 1:sample_size))

ggplot() +
geom_line(aes(x = sample, y = V1, color = type)) +
geom_ribbon(aes(x = sample, ymin = `X2.5.`, ymax = `X97.5.`, fill = type), alpha = 0.2) +
theme_cowplot() +
ylab("Richness") + xlab("Sample") +
theme(legend.title = element_blank()) +

```

```

theme(legend.position = c(0.05, 0.9))

### mean ratio

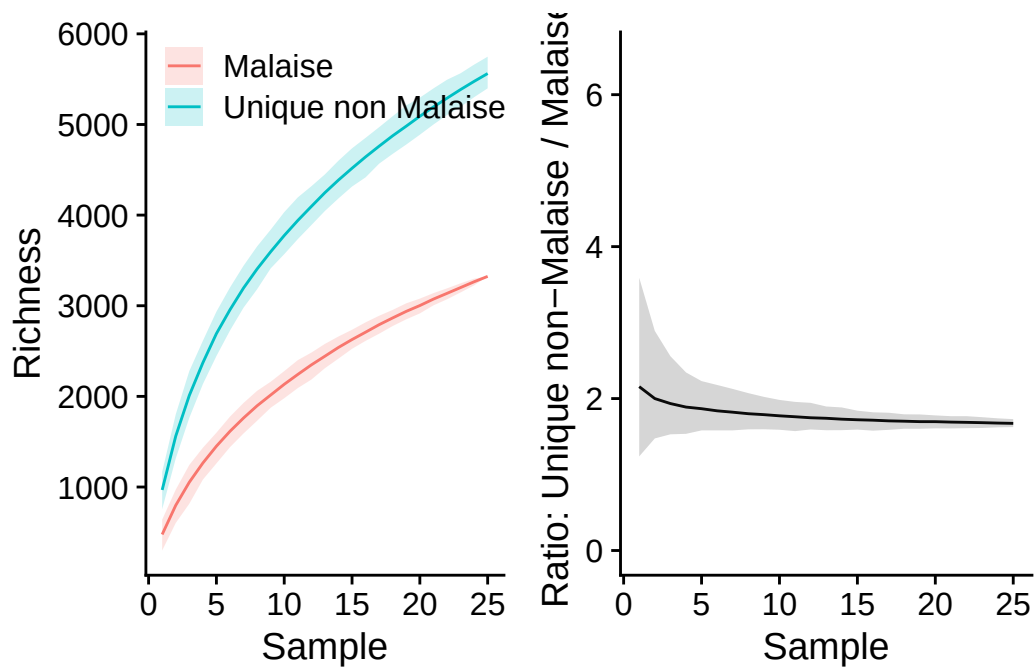
ratio_mean <- apply(accumulated_unique_non_malaise/accumulated_malaise, 1,
  FUN= function(x) c(mean(x), quantile(x, 0.025),
    quantile(x, 0.975)))

ratio_sample_all_fig <- data.frame(t(ratio_mean), sample = 1:sample_size) %>%
  ggplot() +
  geom_line(aes(x = sample, y = V1)) +
  geom_ribbon(aes(x = sample, ymin = `X2.5.`, ymax = `X97.5.`), alpha = 0.2) +
  theme_cowplot() +
  ylab("Ratio: Unique non-Malaise / Malaise") + xlab("Sample") +
  scale_y_continuous(limits = c(0, 6.5))

combined_saturation_ratio <- plot_grid(sample_malaise_non_malaise_fig, ratio_sample_all_fig)

combined_saturation_ratio

```



```
ggsave(combined_saturation_ratio,
       filename = paste0("../Figures/combined_saturation_ratio",sample_size,".jpeg"), width = 10, height = 10)
t(ratio_mean)
```

		2.5%	97.5%
[1,]	2.16	1.24	3.59
[2,]	2.00	1.48	2.89
[3,]	1.93	1.53	2.56
[4,]	1.89	1.54	2.34
[5,]	1.87	1.58	2.23
[6,]	1.84	1.58	2.18
[7,]	1.82	1.58	2.13
[8,]	1.80	1.59	2.07
[9,]	1.79	1.60	2.02
[10,]	1.77	1.59	1.98
[11,]	1.76	1.57	1.96
[12,]	1.75	1.59	1.94
[13,]	1.74	1.58	1.90
[14,]	1.73	1.58	1.88
[15,]	1.72	1.59	1.84
[16,]	1.72	1.58	1.82
[17,]	1.71	1.59	1.81
[18,]	1.70	1.60	1.79
[19,]	1.70	1.60	1.79
[20,]	1.70	1.61	1.78
[21,]	1.69	1.61	1.77
[22,]	1.69	1.61	1.77
[23,]	1.68	1.61	1.76
[24,]	1.68	1.62	1.74
[25,]	1.67	1.62	1.73

Calculate the average unique non Malaise richness by order

```
rownames(accumulated_unique_non_malaise_order) <- orders

apply(accumulated_unique_non_malaise_order, 1, FUN =
      function(x) c(mean(x), quantile(x, 0.025), quantile(x, 0.975))) %>%
  t() %>%
  as.data.frame() %>%
  rename(mean_final_total_bins = V1)
```

	mean_final_total_bins	2.5%	97.5%
Archaeognatha	0.45	0.0	1.0
Blattodea	80.04	67.5	91.5
Coleoptera	844.81	803.4	885.5
Dermaptera	2.80	2.0	4.0
Diptera	1822.00	1731.5	1929.4
Embioptera	0.98	1.0	1.0
Ephemeroptera	1.41	1.0	2.0
Hemiptera	293.94	270.0	317.6
Hymenoptera	355.87	328.9	381.2
Lepidoptera	1946.32	1872.3	2022.7
Mantodea	2.81	2.0	3.0
Neuroptera	10.53	7.0	14.0
Odonata	2.33	0.0	4.0
Orthoptera	36.91	30.5	43.0
Psocodea	100.89	88.5	115.0
Strepsiptera	1.38	0.0	2.0
Thysanoptera	23.49	19.5	28.0
Trichoptera	34.64	31.0	39.5
Zoraptera	0.50	0.0	1.0